
CityShare

Plateforme collaborative de transport de personnes
et de marchandises inter-localités au Cameroun

Rapport de projet tutoré

Auteurs : EBESSA JOSEPHINE HOT CLARISSE
MBARGA ERNEST
MOTCHEYO MENOUNGA GRACE LIANDRA
NGONGANG TCHINCHUT JONATHAN BRYAN
OTTOU VOUNDI JULIE SAMIRA

Encadrant : EVARIS FOMEKONG

Filière : Tronc Commun

Année : 2025 – 2026

Résumé

CityShare est une plateforme mobile collaborative conçue pour répondre aux défis du transport inter-localités au Cameroun. Elle connecte des conducteurs, des passagers et des expéditeurs de colis sur des trajets communs reliant les grandes villes comme Yaoundé et Douala aux localités secondaires telles que Ntui, Obala et Bafia. Le projet propose un modèle hybride unique associant covoiturage et livraison collaborative (*crowd-shipping*), sécurisé par un système multicouche intégrant vérification d'identité, QR code, bouton SOS et *dead man's switch*. Développé avec React Native, Spring Boot et PostgreSQL sans recours à aucune API payante, CityShare vise à structurer un marché du transport informel encore peu adressé par les solutions numériques existantes.

Mots-clés : covoiturage, crowd-shipping, transport inter-localités, Cameroun, sécurité mobile.

Abstract

CityShare is a collaborative mobile platform designed to address transportation challenges between cities and secondary localities in Cameroon. It connects drivers, passengers, and parcel senders on shared trips linking major cities such as Yaoundé and Douala to smaller towns including Ntui, Obala, and Bafia. The project offers a unique hybrid model combining ride-sharing and crowd-shipping, secured through a multi-layer system featuring identity verification, QR codes, an SOS button, and a dead man's switch. Built with React Native, Spring Boot, and PostgreSQL with no reliance on paid APIs, CityShare aims to bring structure and safety to an informal transport market largely overlooked by existing digital solutions.

Keywords : ride-sharing, crowd-shipping, inter-city transport, Cameroon, mobile security.

Table des figures

2.1	Diagramme de cas d'utilisation de CityShare	9
2.2	Diagramme de classes simplifié de CityShare	10
2.3	Diagramme d'états-transitions d'un trajet	11
2.4	Diagramme d'activités — réservation d'une place	12
3.1	Vue d'ensemble des écrans basse-fidélité de CityShare	15
3.2	Vue d'ensemble des écrans haute-fidélité de CityShare	16
A.1	Lean Canvas de CityShare	II

Liste des tableaux

1.1	Comparaison des solutions existantes avec CityShare	5
2.1	Exigences fonctionnelles principales	6
2.2	Exigences non fonctionnelles	7
3.1	Comparaison des frameworks frontend mobiles	13
3.2	Comparaison des frameworks backend	14
3.3	Comparaison des systèmes de gestion de bases de données	14
3.4	Principaux endpoints de l'API CityShare	15

Liste des abréviations

API	Application Programming Interface
CNI	Carte Nationale d'Identité
FCFA	Franç CFA
FCM	Firebase Cloud Messaging
GPS	Global Positioning System
JWT	JSON Web Token
MVP	Minimum Viable Product
OSM	OpenStreetMap
QR	Quick Response
REST	Representational State Transfer
SQL	Structured Query Language
UML	Unified Modeling Language
UI	User Interface
UX	User Experience

Glossaire

Covoiturage Pratique consistant à partager un véhicule entre plusieurs personnes effectuant le même trajet, permettant de réduire les coûts et l’empreinte carbone.

Crowd-shipping Modèle de livraison collaboratif dans lequel des particuliers effectuant un trajet acceptent de transporter des colis pour le compte d’expéditeurs.

Dead man’s switch Mécanisme de sécurité déclenchant automatiquement une alerte si un utilisateur n’interagit pas avec l’application dans un délai défini.

Parrainage communautaire Système selon lequel un nouvel utilisateur doit être recommandé par des membres existants de confiance avant d’être admis sur la plateforme.

Table des matières

Résumé	i
Abstract	ii
Liste des figures	iii
Liste des tableaux	iv
Liste des abréviations	v
Glossaire	vi
Introduction générale	2
1 Concepts et état de l’art	3
1.1 Concepts fondamentaux	3
1.1.1 Application mobile	3
1.1.2 API REST	3
1.1.3 Covoiturage (<i>ride-sharing</i>)	3
1.1.4 Crowd-shipping (livraison collaborative)	4
1.1.5 Sécurité des systèmes distribués	4
1.2 État de l’art	4
1.2.1 Solutions existantes au Cameroun	4
1.2.2 Solutions existantes à l’international	4
1.3 Bilan comparatif	5
2 Méthodologie et conception	6
2.1 Analyse fonctionnelle	6
2.1.1 Acteurs du système	6
2.1.2 Exigences fonctionnelles	6
2.1.3 Exigences non fonctionnelles	7
2.2 Conception UML	7
2.2.1 Diagramme de cas d’utilisation	7

2.2.2	Diagramme de classes	7
2.2.3	Diagramme d'états-transitions	7
2.2.4	Diagramme d'activités	7
2.3	Architecture du système	8
3	Implémentation et résultats	13
3.1	Choix des technologies	13
3.1.1	Framework frontend : React Native vs Flutter vs Ionic	13
3.1.2	Framework backend : Spring Boot vs Django vs Node.js	13
3.1.3	Base de données : PostgreSQL vs MySQL vs MongoDB	13
3.2	Résultats obtenus	14
3.2.1	Prototype basse-fidélité	14
3.2.2	Prototype haute-fidélité	14
3.2.3	Endpoints API implémentés	14
3.2.4	État de l'application	14
	Conclusion générale	17
	Références bibliographiques	I
	A Lean Canvas de CityShare	II
	B Références GitHub	III

Introduction générale

Contexte. Le Cameroun dispose d'un réseau de transport terrestre en pleine mutation, mais les localités secondaires demeurent largement enclavées. Plus de 60 % des trajets entre villes secondaires s'effectuent encore via des véhicules informels sans garantie tarifaire ni sécuritaire. Les solutions numériques de mobilité comme Uber ou Bolt, bien qu'implantées dans les métropoles, n'adressent pas les axes inter-localités tels que Yaoundé–Ntui ou Douala–Bafia.

Problématique. Comment concevoir et développer une plateforme numérique fiable, sécurisée et économiquement viable permettant de structurer le transport collaboratif de personnes et de marchandises entre villes et localités secondaires dans le contexte camerounais ?

Motivation. Ce projet naît d'un constat quotidien : des véhicules circulent avec des sièges vides sur ces axes tandis que des voyageurs peinent à trouver un transport sûr et abordable. La convergence du covoiturage et de la livraison collaborative sur un même trajet représente une opportunité inédite de valeur partagée pour conducteurs, passagers et commerçants.

Objectifs. L'objectif général est de concevoir, prototyper et implémenter le MVP de CityShare, une application mobile de transport collaboratif inter-localités. Plus spécifiquement, il s'agit de : (i) analyser l'état de l'art des solutions existantes et identifier leurs lacunes pour le contexte camerounais ; (ii) concevoir l'architecture logicielle et les modèles UML de la plateforme ; (iii) implémenter les fonctionnalités essentielles du MVP en justifiant chaque choix technologique. Le présent rapport est structuré en trois chapitres : le premier présente les concepts fondamentaux et l'état de l'art, le second détaille la méthodologie et la conception, et le troisième expose l'implémentation et les résultats obtenus.

Chapitre 1

Concepts et état de l’art

1.1 Concepts fondamentaux

1.1.1 Application mobile

Une application mobile est un logiciel conçu pour s’exécuter sur des appareils portables tels que les smartphones et tablettes. Elle se distingue d’une application web par son accès natif aux capteurs du terminal (GPS, appareil photo, notifications push) et sa capacité à fonctionner en mode déconnecté. Dans le cadre de CityShare, l’application mobile constitue l’interface principale par laquelle conducteurs, passagers et expéditeurs interagissent avec la plateforme.

1.1.2 API REST

Une API de type REST (*Representational State Transfer*) est une interface de communication entre composants logiciels reposant sur le protocole HTTP. Elle expose des ressources via des *endpoints* standardisés (GET, POST, PUT, DELETE) et utilise le format JSON pour l’échange de données. CityShare adopte ce paradigme pour toutes les communications entre le frontend React Native et le backend Spring Boot.

1.1.3 Covoiturage (*ride-sharing*)

Le covoiturage désigne le partage d’un véhicule privé entre un conducteur effectuant un trajet pour son propre compte et des passagers souhaitant emprunter le même itinéraire, moyennant une participation aux frais. Ce modèle réduit le coût individuel du transport et le nombre de véhicules sur les routes. CityShare étend ce concept aux localités secondaires camerounaises, un territoire non couvert par les plateformes existantes.

1.1.4 Crowd-shipping (livraison collaborative)

Le crowd-shipping est un modèle logistique dans lequel des particuliers en déplacement acceptent de transporter des colis pour le compte d'expéditeurs sur leur itinéraire habituel. Il s'oppose aux modèles de livraison traditionnels en mutualisant des capacités de transport existantes. CityShare intègre cette fonctionnalité directement dans le flux de création de trajet du conducteur, permettant d'optimiser chaque voyage.

1.1.5 Sécurité des systèmes distribués

Dans un système distribué, la sécurité implique l'authentification des utilisateurs, le chiffrement des communications et le contrôle des accès aux ressources. CityShare implémente un mécanisme d'authentification par JWT (JSON Web Token), dans lequel chaque requête API est accompagnée d'un jeton signé prouvant l'identité de l'émetteur et son rôle sur la plateforme.

1.2 État de l'art

1.2.1 Solutions existantes au Cameroun

Yango (Cameroun). Yango est une application de VTC opérée par Yandex, disponible à Yaoundé et Douala. Elle propose des courses en temps réel avec tarification dynamique. Son périmètre géographique se limite aux zones urbaines denses et elle n'intègre aucune fonctionnalité de livraison de colis ni de transport inter-localités.

Agences de voyage traditionnelles. Les agences telles que Buca Voyages, General Express ou Touristique Express assurent des liaisons entre grandes villes, mais leurs dessertes s'arrêtent aux chefs-lieux de département. L'absence d'interface numérique de réservation et l'inexistence de service vers Ntui, Obala ou Bafia constituent des lacunes majeures.

Groupes WhatsApp informels. Des groupes WhatsApp locaux organisent ponctuellement des covoiturages. Ce modèle manque de structuration, de mécanisme de confiance et de système de paiement intégré.

1.2.2 Solutions existantes à l'international

BlaBlaCar (France). BlaBlaCar est le leader mondial du covoiturage longue distance, présent dans plus de 22 pays. La plateforme permet la réservation de places

sur des trajets publiés à l'avance et intègre un système de notation bilatérale. Elle ne propose pas de livraison de colis et n'est pas déployée en Afrique subsaharienne.

Uber Freight / Uber Connect (USA). Uber Connect permet l'envoi de colis via des conducteurs Uber. Ce service n'est disponible que dans certaines grandes villes américaines et repose sur une infrastructure d'API payantes.

Sendy (Kenya). Sendy est une plateforme africaine de livraison opérant au Kenya, en Ouganda et en Tanzanie. Elle cible principalement les entreprises (B2B) et ne propose pas de covoiturage.

Yassir (Algérie). Yassir est une super-application nord-africaine intégrant VTC, livraison et services financiers. Son modèle est adapté à des marchés à fort taux de bancarisation et ne correspond pas aux réalités camerounaises.

1.3 Bilan comparatif

Le tableau 1.1 présente une comparaison systématique des solutions identifiées.

TABLE 1.1 – Comparaison des solutions existantes avec CityShare

Critère	Yango	BlaBlaCar	Sendy	Yassir	Gp. WA	CityShare
Transport de personnes	Oui	Oui	Non	Oui	Oui	Oui
Livraison de colis	Non	Non	Oui	Oui	Non	Oui
Trajets inter-localités secondaires	Non	Non	Non	Non	Oui	Oui
Arrêts intermédiaires flexibles	Non	Oui	Non	Non	Non	Oui
Paiement Mobile Money	Non	Non	Non	Non	Non	Oui
Vérification CNI conducteur	Non	Non	Non	Oui	Non	Oui
Bouton SOS intégré	Non	Non	Non	Non	Non	Oui
Dead man's switch	Non	Non	Non	Non	Non	Oui
Parrainage communautaire	Non	Non	Non	Non	Oui	Oui
Disponible au Cameroun	Oui	Non	Non	Non	Oui	Oui
Sans API payante (MVP)	Non	Non	Non	Non	Oui	Oui

Au vu de ce tableau comparatif, aucune des solutions répertoriées ne remplit l'ensemble des critères identifiés comme essentiels pour le contexte camerounais. CityShare se positionne comme la seule plateforme combinant covoiturage, crowd-shipping, couverture des localités secondaires et sécurité multicouche adaptée à ce marché.

Chapitre 2

Méthodologie et conception

2.1 Analyse fonctionnelle

2.1.1 Acteurs du système

CityShare implique trois catégories d'utilisateurs :

- **Conducteur** : publie des trajets, fixe le prix, accepte passagers et colis ;
- **Passager** : recherche et réserve une place sur un trajet existant ;
- **Expéditeur** : propose un colis à livrer, un conducteur l'accepte.

2.1.2 Exigences fonctionnelles

TABLE 2.1 – Exigences fonctionnelles principales

ID	Exigence	Priorité
EF01	Un conducteur peut créer un trajet avec départ, arrivée et arrêts intermédiaires	Haute
EF02	Un passager peut rechercher un trajet par ville de départ et d'arrivée	Haute
EF03	Un expéditeur peut proposer un colis sur un trajet existant	Haute
EF04	Le système génère un QR code unique par réservation	Haute
EF05	Le passager peut déclencher une alerte SOS envoyant sa position	Haute
EF06	Le dead man's switch envoie une alerte si l'arrivée n'est pas confirmée	Moyenne
EF07	Le système notifie les utilisateurs par notification push	Moyenne
EF08	Les utilisateurs peuvent s'évaluer mutuellement après chaque trajet	Moyenne

TABLE 2.2 – Exigences non fonctionnelles

Catégorie	Description
Performance	Le temps de réponse de l'API ne doit pas excéder 2 secondes pour 95 % des requêtes
Sécurité	Toutes les communications sont chiffrées via HTTPS ; les tokens JWT expirent après 24 h
Disponibilité	La plateforme doit être disponible 99 % du temps hors fenêtres de maintenance
Scalabilité	L'architecture doit supporter une montée en charge de 0 à 5 000 utilisateurs sans refonte
Utilisabilité	L'application doit être utilisable sans formation préalable par un utilisateur non initié
Maintenabilité	Le code source est versionné sur GitHub avec branches <code>main</code> et <code>dev</code> séparées

2.1.3 Exigences non fonctionnelles

2.2 Conception UML

Les diagrammes UML ont été réalisés avec l'outil draw.io et sont présentés dans les figures suivantes.

2.2.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation (figure 2.1) illustre les interactions entre les trois acteurs (Conducteur, Passager, Expéditeur) et les fonctionnalités principales de la plateforme : publication de trajet, réservation, proposition de colis, déclenchement SOS et gestion du profil.

2.2.2 Diagramme de classes

Le diagramme de classes (figure 2.2) représente les entités principales du domaine : Utilisateur, Trajet, Réservation, Colis et Notation, ainsi que leurs relations.

2.2.3 Diagramme d'états-transitions

Le diagramme d'états-transitions (figure 2.3) représente les états possibles d'un trajet : Brouillon, Actif, Complet, En cours, Terminé et Annulé.

2.2.4 Diagramme d'activités

Le diagramme d'activités (figure 2.4) illustre le flux complet lors de la réservation d'une place par un passager.

2.3 Architecture du système

CityShare adopte une architecture trois-tiers classique :

- **Couche présentation** : application React Native communiquant exclusivement via des appels REST API ;
- **Couche métier** : backend Spring Boot exposant les endpoints API, gérant la logique de trajet, de réservation et de livraison ;
- **Couche données** : base PostgreSQL stockant l'ensemble des entités du domaine.

Chaque requête API est interceptée par un filtre Spring Security qui vérifie la validité du token JWT. Les rôles (CONDUCTEUR, PASSAGER, EXPÉDITEUR) sont encodés dans le token et contrôlent l'accès aux ressources.



FIGURE 2.1 – Diagramme de cas d'utilisation de CityShare

Diagramme de classe

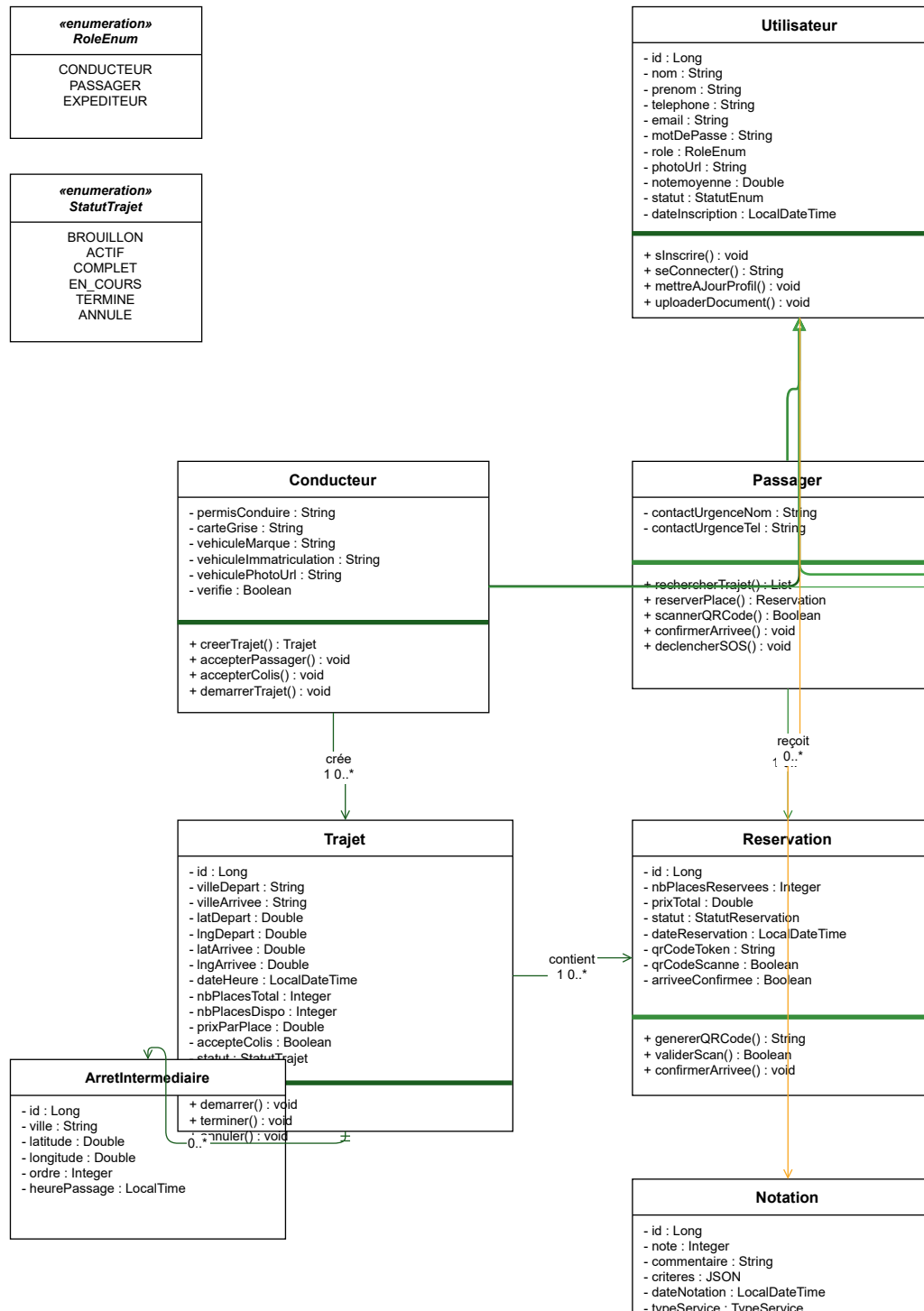


FIGURE 2.2 – Diagramme de classes simplifié de CityShare

Diagramme d'états-transitions — Cycle de vie d'un Trajet (CityShare)

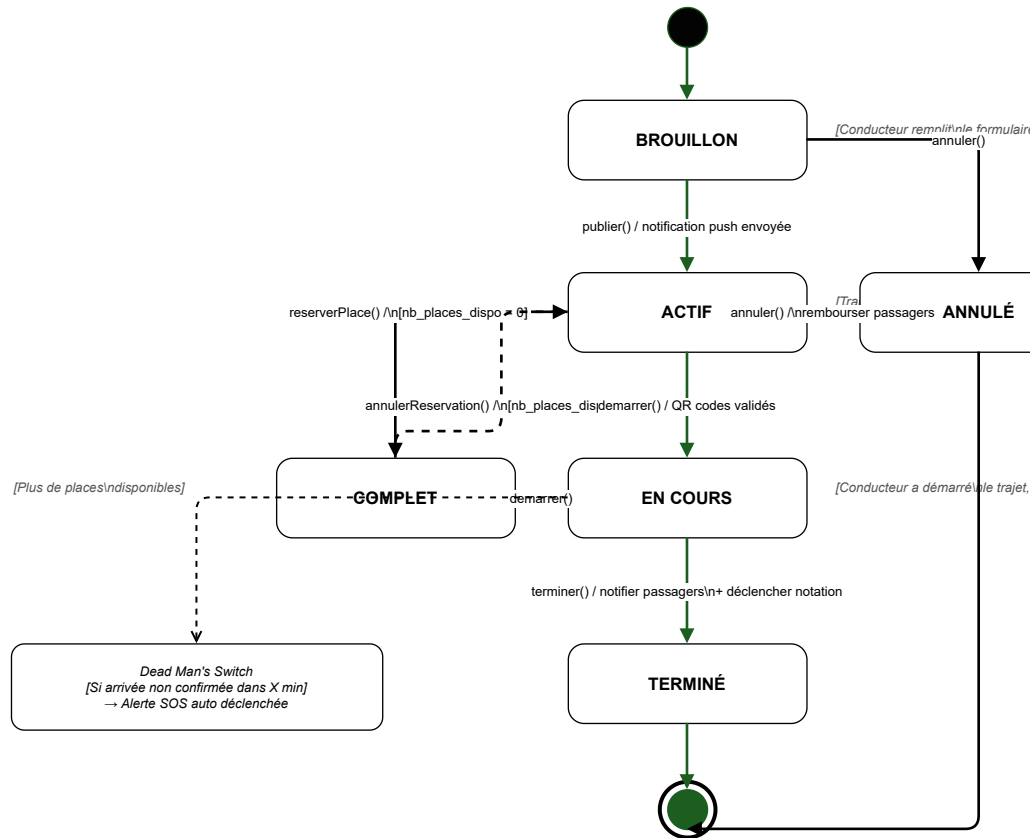


FIGURE 2.3 – Diagramme d'états-transitions d'un trajet

Diagramme d'activités — Réservation d'une place (CityShare)

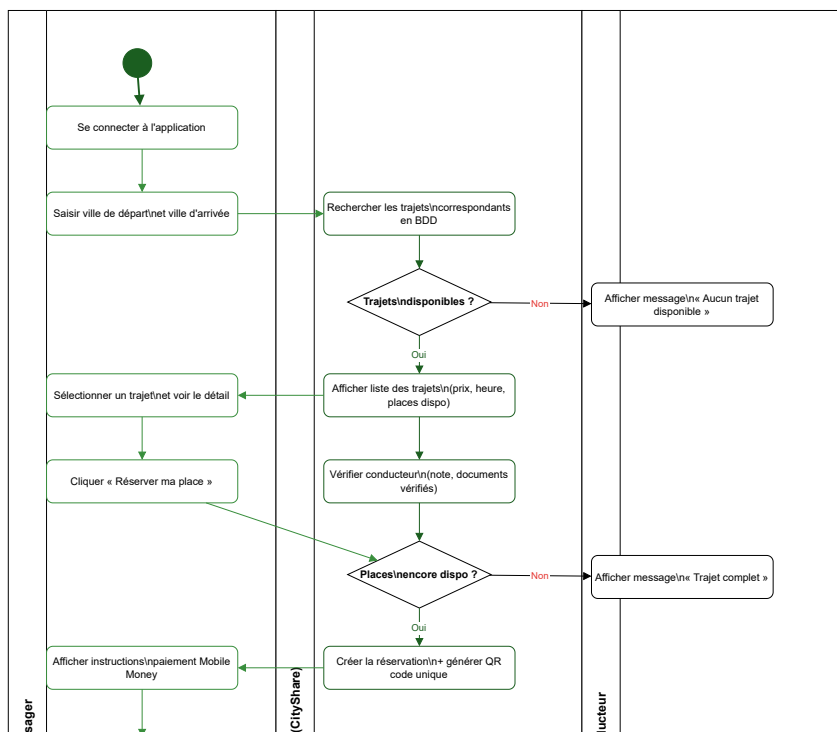


FIGURE 2.4 – Diagramme d'activités — réservation d'une place

Chapitre 3

Implémentation et résultats

3.1 Choix des technologies

3.1.1 Framework frontend : React Native vs Flutter vs Ionic

TABLE 3.1 – Comparaison des frameworks frontend mobiles

Critère	React Native	Flutter	Ionic
Langage	JavaScript/TS	Dart	JavaScript
Performance native	Haute	Très haute	Moyenne
Communauté / Écosystème	Très large	Large	Moyenne
Courbe d'apprentissage	Faible (JS connu)	Moyenne (Dart)	Faible
Accès GPS, caméra, notifications	Oui	Oui	Limité
Déploiement iOS + Android	Oui	Oui	Oui
Choix retenu			

React Native a été retenu pour la maîtrise préalable du langage JavaScript par l'équipe, sa grande communauté et sa capacité à accéder aux fonctionnalités natives (GPS, caméra, notifications) nécessaires à CityShare.

3.1.2 Framework backend : Spring Boot vs Django vs Node.js

Spring Boot est retenu pour son écosystème de sécurité mature (Spring Security + JWT), son support natif des WebSockets via STOMP et sa robustesse éprouvée en environnement de production.

3.1.3 Base de données : PostgreSQL vs MySQL vs MongoDB

PostgreSQL est sélectionné pour son support natif des données géospatiales via l'extension PostGIS, indispensable pour la gestion des trajets et arrêts intermédiaires.

TABLE 3.2 – Comparaison des frameworks backend

Critère	Spring Boot	Django	Node.js
Langage	Java	Python	JavaScript
Sécurité intégrée (JWT)	Excellente	Bonne	Manuelle
ORM intégré	Spring Data JPA	Django ORM	Sequelize
Performance	Haute	Moyenne	Haute
WebSockets natifs	Oui (STOMP)	Partiel	Oui
Communauté entreprise	Très forte	Forte	Forte
Choix retenu			

TABLE 3.3 – Comparaison des systèmes de gestion de bases de données

Critère	PostgreSQL	MySQL	MongoDB
Modèle de données	Relationnel	Relationnel	Document
Transactions ACID	Complètes	Complètes	Partielles
Support géospatial (trajets)	Excellent (PostGIS)	Limité	Limité
Hébergement gratuit	Render, Railway	Limité	Atlas (limité)
Intégration Spring Data	Native	Native	Native
Choix retenu			

3.2 Résultats obtenus

3.2.1 Prototype basse-fidélité

Les maquettes basse-fidélité ont été réalisées avec Balsamiq et couvrent les écrans principaux de la plateforme : onboarding, recherche de trajet, réservation et écran SOS. La figure 3.1 présente une vue d'ensemble de ces maquettes.

3.2.2 Prototype haute-fidélité

Les maquettes haute-fidélité ont été réalisées sur Figma et couvrent l'ensemble du parcours utilisateur : onboarding, inscription avec sélection de rôle, recherche de trajet, détail de trajet avec QR code, tableau de bord conducteur, envoi de colis et écran SOS. La figure 3.2 présente une vue d'ensemble des écrans du prototype.

3.2.3 Endpoints API implémentés

3.2.4 État de l'application

À la date de rédaction du présent rapport, le serveur backend est déployé sur Render (offre gratuite) et accessible via l'URL <https://cityshare-api.onrender.com>. L'application React Native se connecte à ce backend via des appels REST authentifiés par JWT. Les fonctionnalités de gestion des utilisateurs, de création et de recherche

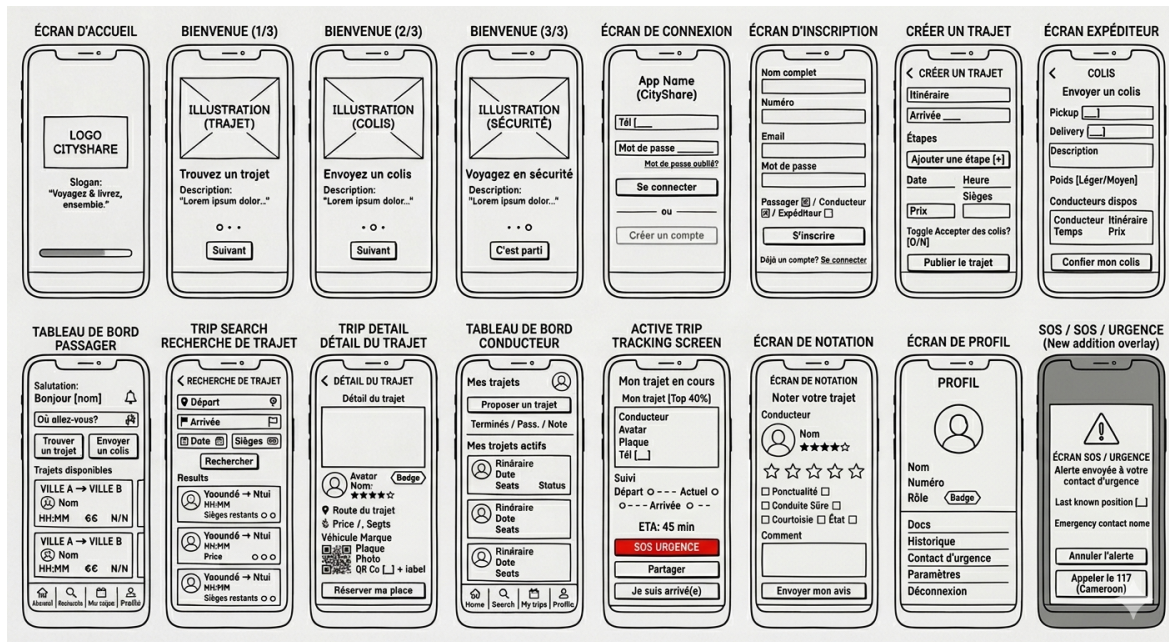


FIGURE 3.1 – Vue d'ensemble des écrans basse-fidélité de CityShare

TABLE 3.4 – Principaux endpoints de l'API CityShare

Méthode	Endpoint	Description
POST	/api/auth/register	Inscription d'un nouvel utilisateur
POST	/api/auth/login	Connexion et génération du token JWT
POST	/api/trajets	Création d'un nouveau trajet (conducteur)
GET	/api/trajets	Recherche de trajets disponibles
POST	/api/reservations	Réservation d'une place
GET	/api/reservations/{id}/qr	Récupération du QR code
POST	/api/colis	Proposition d'un colis
POST	/api/sos	Déclenchement d'une alerte SOS
POST	/api/notations	Soumission d'une notation post-trajet

de trajets, ainsi que le système de QR code sont fonctionnels.

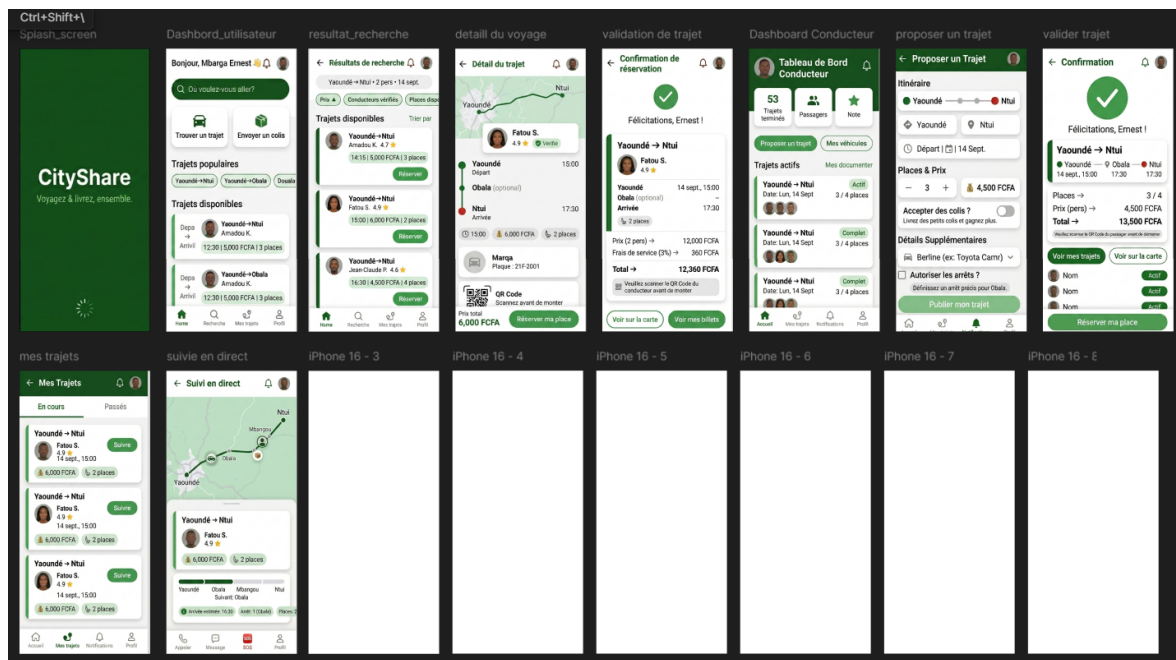


FIGURE 3.2 – Vue d'ensemble des écrans haute-fidélité de CityShare

Conclusion générale

Ce rapport a présenté la conception et le développement de CityShare, une plateforme collaborative de transport inter-localités adaptée au contexte camerounais. L'analyse de l'état de l'art a confirmé l'absence de toute solution numérique structurée répondant aux besoins des axes reliant les grandes villes aux localités secondaires comme Ntui, Obala ou Bafia. La méthodologie adoptée, articulée autour de l'analyse fonctionnelle et non fonctionnelle, des diagrammes UML et d'une architecture trois-tiers sécurisée, a permis de poser des fondations solides pour le développement. Le MVP implémenté démontre la faisabilité technique du projet avec des technologies éprouvées et sans dépendance à des API payantes.

Les perspectives à court terme incluent l'intégration d'un système de paiement Mobile Money réel (Orange Money, MTN Mobile Money), l'extension de la couverture géographique à de nouvelles localités, et la mise en place d'une campagne d'acquisition dans les universités de Yaoundé. À moyen terme, CityShare ambitionne de devenir la référence du transport collaboratif en Afrique centrale.

Bibliographie

- [1] Ministère des Transports du Cameroun, *Rapport annuel sur la mobilité inter-urbaine et les transports en commun*, Yaoundé, 2023.
- [2] A. Charland et B. Leroux, “Mobile application development : web vs. native,” *Communications of the ACM*, vol. 54, no. 5, pp. 49–53, 2011.
- [3] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, Thèse de doctorat, Univ. of California, Irvine, 2000.
- [4] M. Furuhashi et al., “Ridesharing : The state-of-the-art and future directions,” *Transportation Research Part B*, vol. 57, pp. 28–46, 2013.
- [5] K. Gdowska et al., “Stochastic last-mile delivery with crowdshipping,” *Transportation Research Procedia*, vol. 30, pp. 90–100, 2018.
- [6] W. Stallings, *Cryptography and Network Security*, 7^e éd. Pearson, 2017.
- [7] BlaBlaCar, *Long-distance carpooling platform*, 2023. [En ligne] <https://www.blablacar.fr>
- [8] Sendy, *African last-mile logistics*, 2023. [En ligne] <https://sendy.co.ke>
- [9] Yango, *Ride-hailing service Cameroun*, 2024. [En ligne] https://yango.com/fr_cm/

Annexe A

Lean Canvas de CityShare

Le Lean Canvas (figure A.1) synthétise le modèle économique et stratégique de CityShare : problèmes adressés, solution proposée, segments clients, canaux de distribution et sources de revenus.

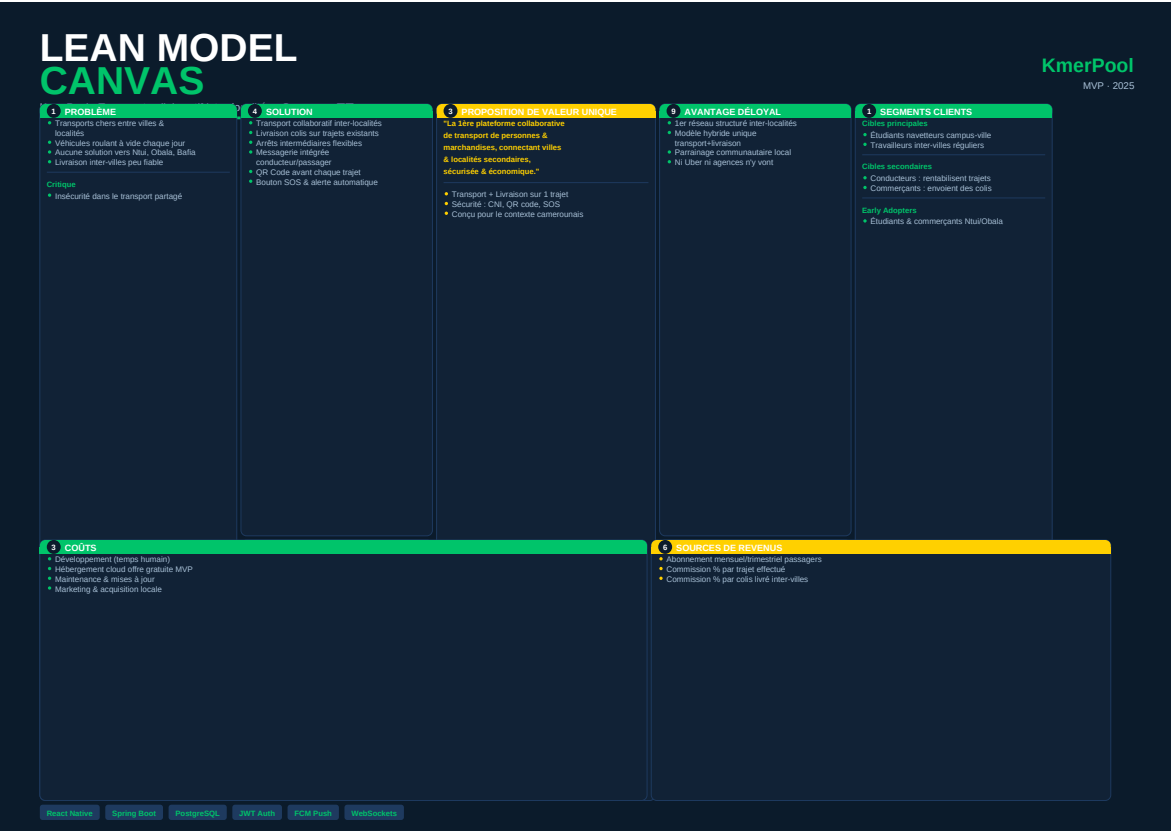


FIGURE A.1 – Lean Canvas de CityShare

Annexe B

Références GitHub

- Frontend React Native : <https://github.com/kiyotaka-sudo/cityshare-frontend>
- Backend Spring Boot : <https://github.com/kiyotaka-sudo/cityshare-backend>
- Branches : `main` (production) et `dev` (développement)